

System.Linq practice tasks

Five short console exercises, from a simple filter to a grouped join.

Open LinqPractice.sln, in the same folder as this document. Each task is its own console project. The sample data is already in the project. Complete the Solve method and run the project.

A finished method is one LINQ query, or a few operators around one. Method syntax and query syntax are both fine. Do not change the sample data, the records, or the checker.

How to run a task

- From this folder: `dotnet run --project Task01.FilterAndSort`
- Or set that project as the startup project and run it from the IDE.

The program prints your result, then Correct or Not yet correct. Until Solve returns a value, it prints a short reminder instead. The checker compares the printed text with the expected result shown in each task. That expected text is not stored in the source.

What increases

- Task 1 filters, sorts, and projects a single sequence.
- Task 2 reduces one filtered sequence to a count, a sum, an average, and a tie-break.
- Task 3 groups rows and aggregates each group.
- Task 4 inner-joins two sequences, then filters and sorts the pairs.
- Task 5 joins, groups, and groups again, then keeps groups that pass two tests.

Task 1

In-stock books

Project: Task01.FilterAndSort

Operators to practice: Where, OrderBy, ThenBy, Select

Each product has a name, a category, a price, and an in-stock flag. Return the names of products that meet all three conditions:

- Category is Book
- the product is in stock
- Price is less than 25

Sort by price from lowest to highest. When two prices are equal, sort by name from A to Z. Return `IEnumerable<string>`. The program prints one name per line.

Products

Name	Category	Price	In stock
The Hobbit	Book	12.50	yes
Dune	Book	18.00	yes
Notebook	Stationery	4.00	yes
Foundation	Book	22.00	no
Neuromancer	Book	9.99	yes
Pen	Stationery	1.50	yes
Snow Crash	Book	18.00	yes

Expected output

```
Neuromancer
The Hobbit
Dune
Snow Crash
```

Hint. Filter with Where, sort with OrderBy and ThenBy, then Select the name. Dune and Snow Crash share a price, so the second sort decides their order.

Task 2

Sales payroll

Project: Task02.Aggregates

Operators to practice: Where, Count, Sum, Average, OrderBy, First

Employees have a name, a department, and a salary. Build one PayrollSnapshot for the Sales department only:

- Count: how many Sales employees there are
- Total: sum of their salaries
- Average: average salary, rounded to 2 decimal places, with the midpoint rounded away from zero
- LowestPaid: name of the Sales employee with the lowest salary. If several share that salary, choose the name that comes first alphabetically.

Return a PayrollSnapshot. The program prints it in the form shown below.

Employees

Name	Department	Salary
Ann	Sales	52000
Ben	Engineering	90000
Cara	Sales	48000
Dan	Sales	61000
Eve	Engineering	85000
Finn	Sales	48000

Expected output

```
Count: 4
Total: 209000.00
Average: 52250.00
Lowest paid: Cara
```

Hint. Restrict the sequence to Sales before you aggregate. For LowestPaid, sort by salary and then by name, and take the first name. Math.Round(value, 2, MidpointRounding.AwayFromZero) matches the rounding rule.

Task 3

Agent workload

Project: Task03.Grouping

Operators to practice: GroupBy, Count, Sum, OrderByDescending, ThenBy

Each ticket has an id, an agent, a priority (Low, Medium, or High), and hours spent. For every agent, build an AgentLoad:

- Agent
- Tickets: number of tickets
- Hours: sum of hours
- HighPriority: number of tickets whose priority is High

Sort by hours from highest to lowest. When hours are equal, sort by agent name from A to Z.

Tickets

Id	Agent	Priority	Hours
1	Alex	High	3
2	Blair	Medium	2
3	Alex	Low	1
4	Casey	High	5
5	Blair	High	4
6	Alex	High	2
7	Casey	Medium	1
8	Blair	Low	2

Expected output

```
Blair | tickets 3 | hours 8 | high 1
Alex | tickets 3 | hours 6 | high 2
Casey | tickets 2 | hours 6 | high 1
```

Hint. GroupBy the agent. Count() is the ticket total. A second Count with a predicate counts the High tickets. Alex and Casey logged the same hours, so the name decides who comes first.

Task 4

Orders in two cities

Project: Task04.Joins

Operators to practice: Join, Where, OrderBy, ThenBy, ThenByDescending

Customers and orders are separate sequences. Join them on the customer id. Keep a row only when both are true:

- the customer city is Oslo or Bergen
- the order amount is at least 100

Return an OrderLine with the customer name, city, product, and amount. Sort by city from A to Z, then by customer name from A to Z, then by amount from highest to lowest.

Customers

Id	Name	City
1	Nora	Oslo
2	Lars	Bergen
3	Mia	Trondheim
4	Erik	Oslo

Orders

Id	Customer id	Product	Amount
101	1	Keyboard	120
102	1	Mouse	40
103	2	Monitor	300
104	3	Desk	250
105	4	Chair	180
106	2	Lamp	90
107	4	Headset	150
108	1	Webcam	100

Expected output

```
Lars | Bergen | Monitor | 300
Erik | Oslo | Chair | 180
Erik | Oslo | Headset | 150
Nora | Oslo | Keyboard | 120
Nora | Oslo | Webcam | 100
```

Hint. Join first, so the filter can see both the city and the amount. Leave out Mia, who lives in Trondheim, and leave out amounts under 100. Bergen comes before Oslo alphabetically.

Task 5**Genres under strain**

Project: Task05.Genres

Operators to practice: Join, GroupBy, Distinct, a nested GroupBy, Where, OrderByDescending

Loans and books are separate sequences. A loan stores a book id, a member name, and days overdue. Zero means the book was returned on time.

Join loans to books, then group by genre. Produce a GenreReport only when both are true:

- at least 2 different members have a loan in that genre
- the average days overdue is greater than 1

Average = (sum of DaysOverdue for loans in the genre) / (number of loans in the genre), then rounded to 2 decimal places, midpoint away from zero.

MostOverdueBook is the title in that genre with the greatest sum of DaysOverdue. If two titles have the same sum, choose the title that comes first alphabetically.

Sort the reports by average overdue from highest to lowest. When averages are equal, sort by genre from A to Z.

Books

Id	Title	Genre
1	Dune	SciFi
2	Neuromancer	SciFi
3	Foundation	SciFi
4	Persuasion	Classic
5	Emma	Classic
6	Pride and Prejudice	Classic
7	The Hobbit	Fantasy

Loans

Book id	Member	Days overdue
1	Ana	0
1	Ben	4
2	Ana	2
2	Cal	0
3	Ben	3
3	Ana	6
4	Ben	3
4	Ana	1
5	Cal	4
6	Ben	0
6	Cal	0
7	Cal	0

Expected output

```
SciFi | members 3 | avg 2.50 | Foundation  
Classic | members 3 | avg 1.60 | Emma
```

Hint. One Join, then GroupBy genre. Member count is the distinct member names. The book title needs a second GroupBy, on title, inside the genre. Apply the two genre tests after those values exist. Fantasy drops out. In Classic, two titles share the highest overdue sum; alphabetical order decides the title.